GROUP EVALUATION REPORT · ASSIGNMENT 2

Collaborative Development and Evidence-Based Evaluation

Structured Bug Report Helper · Assignment 2

AUT · Contemporary Methods in Software Engineering

Prepared 4 November 2026 · Fixed technical review candidate: a2-review-v0.1

Member	Name / student ID / GitHub
A	Pending real team input
B	Pending real team input
C	Pending real team input
D	Pending real team input

Main text: 1889 whitespace words / 2050 conservative tokens; limit 2,500. Tables and citations are included.

Preparation draft. Software and technical experiments are implemented; real member identities, independent reviews, team collaboration records, product peer evaluation, individual statements and assessor access remain pending.

https://github.com/alwaysgoodz/aut-bug-report-helper/tree/feat/a2-iteration-2

1. Increment scope and changes

The increment continues Assignment 1's lecturer-approved structured bug-report project. It now supports seven-field report creation, persistent list/detail views, guarded status changes and reopening, live completeness guidance, and severity/status filtering. The intended users remain small student teams. Completeness reports information presence, not correctness or independent reproducibility; the distinction follows the practical limits of report-element evidence (Soltani et al., 2020). AI diagnosis, automatic fixes, external tracker integration, mobile applications and advanced analytics remain excluded.

Three changes require explanation. A2 places both iterations in Week 2, so the proposed calendar changes from A1's tentative five days each to two working days each plus a final preparation day. The proposed capacity remains 20 team person-hours per iteration, awaiting member confirmation. Process measures are added because the original four-peer product study cannot evaluate a development-workflow change. Playwright is introduced as a locked test-only dependency; Python/SQLite application execution still needs no third-party package.

The two technical increments and experiments below were produced in one AI-assisted preparation session. They are not evidence of a completed four-person Week 2 collaboration. Actual member identities, substantive independent reviews, team retrospectives, individual statements and the planned peer study remain pending. No user-value improvement or human contribution is inferred from generated activity.

2. Lifecycle application and iteration outcomes

The repository preserves the A1 baseline and separate I1/I2 branches. Existing Issues #1–6 supply priorities and acceptance criteria; PI-01 (#9) records the process change after I1. Stacked draft PRs #7, #8 and #10 connect requirements to proposed changes. They remain unmerged pending non-author approval. Proposed roles are A requirements/evaluation, B interaction, C service/storage and D acceptance/CI; names and actual activities must replace these role labels.

Table 1. Technical increment goals, acceptance and outcomes

Increment	Selected work and acceptance	Observed outcome
I1	BR-02/03: create/list safely, retain invalid inputs, persist status cycle/reopening, reject stale or invalid updates.	34 Python/HTTP checks pass; ad-hoc browser C1/C2/C3 pass. Missing-focus mutation escapes automated checks. Snapshot cee24aaa908991a9ff38967bfc2a46f973493934; PR #8.
I2	BR-04/05 and PI-01: exact missing-field guidance, 0/86/100 score boundaries, AND/clear filters, repeatable browser checks and targeted fault detection.	43 functional Python/HTTP checks and seven browser checks pass. Same focus mutation is detected, original restored, full suite passes. Snapshot f36dfc792c4467c0726598cce401629f7a9e7fe5; PR #10.

Each increment records goal, selected requirements, proposed responsibilities, outcomes, decisions and a technical retrospective. Those records distinguish preparation observations from the real team discussions still required. The inherited Definition of Done requires acceptance evidence, latest-revision independent approval and updated documentation; therefore implemented functionality is not labelled fully Done solely because tests pass.

The browser adapter, validation/service, pure completeness rules and SQLite store remain separate. Parameterised queries preserve report text as data; explicit connection closure carries forward A1's Windows lesson. Status updates compare the displayed previous state before writing, so stale pages receive a conflict rather than silently overwriting another change. The repository supports development collaboration, while the product remains one local instance rather than a shared remote database.

3. Quality results and reproducibility

Functional checks exercise valid and rejected creation, separate-process persistence, field limits, status transitions/conflicts, exact omissions and non-mutating filters. Browser checks additionally execute JavaScript, keyboard form submission, narrow-screen layout and safe rendering. Local unittest discovers 44 methods: 43 functional checks pass and the CI-only bridge is skipped. Remote Python 3.12 executes that bridge and all seven browser cases; Python 3.11 skips it. Thus a local skip is not counted as browser evidence.

Local conditions were Windows 11, Anaconda Python 3.12.7, SQLite 3.51.0, Node 22.22.1 and Chrome 155.0.8059.86. Remote run 41680263474 passed on Ubuntu 24.04, Python 3.11.17/3.12.16, Node 22.23.4 and Chromium 156.0.8078.4. The local CDN download failed; installed Chrome was an explicit fallback. The CI log verifies seven passed cases, zero failures/skips/flakiness and the browser version. These environments provide useful diversity, not a full compatibility matrix.

Table 2. Quality scenarios and measured results

Scenario and procedure	Result	Interpretation boundary
Security/integrity: stored markup, CSRF tokens, foreign Origin/Host, bounded requests, duplicate fields, invalid filters and stale writes.	Selected HTTP assertions pass; browser displays injected markup as text, not an image. Rejected writes preserve stored records.	Regression checks within the local threat model; no penetration-test or production-security claim.
Performance: 500 synthetic reports, one loopback client, five warm-ups, 50 full-body list requests. Predeclared nearest-rank p95 target <250 ms.	Median 51.57 ms; p95 77.88 ms; maximum 80.20 ms. Each response 430,353 bytes; all HTTP 200. Target met.	One machine, sequential requests and warm cache. No cold-start, concurrent-user or public-network conclusion.

The performance helper creates its own temporary database and ephemeral server, records every duration, database size, environment and source hashes, and cleans up without touching user data. Measurement used clean implementation f36dfc7. README commands reproduce functional, browser, mutation and latency checks. The fixed technical review tag a2-review-v0.1 preserves the delivered package; the eventual assessed submission still needs team approval and verified assessor access.

Security choices follow token/origin defence guidance (OWASP Foundation, n.d.), but loopback binding does not create user authentication. Python's HTTP server documentation warns against production use (Python Software Foundation, n.d.); this limits the deployment claim. The prototype has no accounts, remote database, body editing, attachments or pagination. Only Chromium-family browsers were exercised. The four-peer completion-time/completeness/reproduction protocol remains unrun, so functionality and latency do not demonstrate usefulness to students.

4. Process improvement and critical interpretation

I1 exposed a specific feedback gap: Python/HTTP tests inspect responses but do not execute client-side focus behaviour. Temporarily disabling the error-summary focus statement left all 34 automated checks green, while an ad-hoc browser observation found the summary visible but unfocused. The statement was restored immediately. This controlled mutation was an experiment, not a delivered defect or a teammate's review finding.

Before I2, the improvement plan fixed three common scenarios: C1 invalid submission retains inputs, inserts nothing and focuses linked errors; C2 creation survives reload without duplication; C3 saved state completes Open/In progress/Resolved/Open. I2 committed these browser assertions and enabled them in CI, retaining faster service/HTTP checks. New guidance, filtering, keyboard and narrow-screen cases extend the suite but are excluded from the shared-scenario comparison.

Table 3. PI-01 observations across the two increments

Measure	I1	I2
Common C1/C2/C3 pass observations	3/3 through an ad-hoc MCP browser exercise	3/3 through committed browser tests; also executed remotely
Common scenarios in committed browser regression/CI	0/3	3/3
Identical missing-focus mutation detected by automated checks	0/1; 34 Python/HTTP checks still pass	1/1; C1 fails at toBeFocused; original bytes restored
Unmodified I2 browser suite after restoration	Not applicable	7/7 pass locally and remotely; zero skipped or flaky cases

This supports a narrow conclusion: the new executable checks detect the chosen client-side regression and make the common scenarios repeatable in CI. It does not establish a general mutation score, fewer escaped defects, improved teamwork or faster delivery. Inozemtseva and Holmes (2014) show why coverage should not be a stand-alone effectiveness target. Just et al. (2014) support studying mutation detection while documenting limits; their broader real-fault evidence does not make one intentionally selected mutant representative.

A1's planned completion time, completeness and independent reproduction assess the product. We retain that study but add executable-scenario coverage and targeted detection for the process question; the change is recorded prospectively. Raw durations are descriptive: I1's ad-hoc exercise took 1.081 seconds, I2's local browser suite 12.382 seconds and remote browser suite 8.414 seconds. Different test counts, startup, machines and browser versions prevent treating these as a productivity or speed improvement.

The experiment also exposed evidence-tooling problems. An anchored test filter matched no test; a JSON result placed inside the HTML reporter's directory was cleared. The helper now verifies the intended assertion fails, writes JSON separately and restores original bytes in finally. The first harness error is preserved rather than counted as detection. Next, the team should test several independently chosen client-side faults and discuss CI cost against risks; a genuine retrospective and a comparable human workload record are still required.

5. Comparative evaluation of approaches and tools

The original selection remains defensible as a small iterative lifecycle with selected agile practices and explicit baselines. I1 produced the core workflow and the observed testing gap; I2 added guidance/filtering and a measured checking improvement. A plan-driven alternative could have specified the browser-focus oracle earlier and reduced this gap; the experience therefore supports stronger up-front acceptance design, not abandoning planning. Conversely, freezing all guidance before evaluating an increment would constrain adaptation without evidence that its wording helps users.

Boehm and Turner (2003) support tailoring practices to risk rather than selecting a universal method. Dybå and Dingsøyr (2008) identify context and evidence-quality limits in early agile research. Here, two technical increments show feasibility and adaptation, but absent stakeholder feedback and human collaboration prevent claims that agile improved team performance. The practical choice is to keep a stable scope, short changes and explicit acceptance contracts while revising a specific checking practice when evidence warrants it.

Table 4. Practices and credible alternatives

Decision	Evidence and merit	Alternative and trade-off
Executable browser checks alongside Python/HTTP	The same focus fault escapes the lower-layer suite and is caught in the browser. Native forms still work without JavaScript.	HTTP-only checks are faster and simpler but miss DOM behaviour. Selenium WebDriver is credible where existing language/grid expertise matters; no Selenium experiment was run.
GitHub CI and tracked PR proposals	Runs bind results to exact commits; local/remote browsers both execute acceptance. OAuth lacks workflow-write scope, so the existing Python 3.12 job invokes the browser gate.	A separate browser job would separate feedback and dependencies; the present bridge couples them. Manual-only checks avoid setup cost but cannot provide the same repeatable gate.
Local Python/SQLite prototype	No runtime package/service setup; temporary test databases isolate experiments. Stale writes are rejected explicitly.	A framework and hosted database would better support authentication or remote users, at additional setup/operational cost. Those requirements are outside this increment.

Playwright's documentation describes executable CI checks and reports (Microsoft, n.d.); Selenium documents WebDriver as another browser-automation approach (Selenium, n.d.). Tool availability is not proof of superior outcomes. Elazhary et al. (2022) show CI implementation depends on context and can constrain feedback; the actual download failure, reporter collision and workflow-scope restriction illustrate our maintenance cost. Tests do not replace thoughtful review: McIntosh et al.'s (2014) observational study links review participation with quality, whereas these unreviewed draft PRs demonstrate only a review opportunity.

For a distributed team, preserve increments but add asynchronous decisions, explicit handovers and review windows; Camara et al.'s (2024) large-scale review informs these adaptations without justifying a scaled framework for four students. A regulated context would require formal risk, approval and requirement-test records and a different deployment assurance level. Those conditions favour more planning/verification around increments; neither our single-user latency nor targeted mutation result transfers directly.

6. Demonstration and remaining team work

The prepared ten-minute runbook demonstrates incomplete/complete creation, invalid-input recovery, persistence, status/reopening, filters, actual CI and the before/after mutation evidence, followed by five minutes of questions. Proposed speaking slots require each member to explain a substantive requirement, design, testing or evaluation decision after performing real work. Individual statement templates request linked decisions and outcomes rather than commit totals.

The software is runnable and its technical observations are reproducible. Before assessed submission, real members must review the latest changes, conduct and record their own iteration discussions, complete the peer evaluation or justify its omission, provide identities and authentic individual statements, verify assessor permissions and select the approved submission reference. The technical review tag is a fixed preparation candidate, not an assertion that these collaborative obligations have been met.

References

Boehm, B., & Turner, R. (2003). Using risk to balance agile and plan-driven methods. *Computer, 36*(6), 57–66. https://doi.org/10.1109/MC.2003.1204376

Camara, R., Marinho, M., & Moura, H. (2024). Agile tailoring in distributed large-scale environments using agile frameworks: A systematic literature review. *CLEI Electronic Journal, 27*(1), Article 8. https://doi.org/10.19153/cleiej.27.1.8

Dybå, T., & Dingsøyr, T. (2008). Empirical studies of agile software development: A systematic review. *Information and Software Technology, 50*(9–10), 833–859. https://doi.org/10.1016/j.infsof.2008.01.006

Elazhary, O., Werner, C., Li, Z. S., Lowlind, D., Ernst, N. A., & Storey, M.-A. (2022). Uncovering the benefits and challenges of continuous integration practices. *IEEE Transactions on Software Engineering, 48*(7), 2570–2583. https://doi.org/10.1109/TSE.2021.3064953

Inozemtseva, L., & Holmes, R. (2014). Coverage is not strongly correlated with test suite effectiveness. In *Proceedings of the 36th International Conference on Software Engineering* (pp. 435–445). Association for Computing Machinery. https://doi.org/10.1145/2568225.2568271

Just, R., Jalali, D., Inozemtseva, L., Ernst, M. D., Holmes, R., & Fraser, G. (2014). Are mutants a valid substitute for real faults in software testing? In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering* (pp. 654–665). Association for Computing Machinery. https://doi.org/10.1145/2635868.2635929

McIntosh, S., Kamei, Y., Adams, B., & Hassan, A. E. (2014). The impact of code review coverage and code review participation on software quality: A case study of the Qt, VTK, and ITK projects. In *Proceedings of the 11th Working Conference on Mining Software Repositories* (pp. 192–201). Association for Computing Machinery. https://doi.org/10.1145/2597073.2597076

Microsoft. (n.d.). *Setting up CI.* Playwright. Retrieved November 4, 2026, from https://playwright.dev/docs/ci-intro

OWASP Foundation. (n.d.). *Cross-site request forgery prevention cheat sheet.* OWASP Cheat Sheet Series. Retrieved November 4, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

Python Software Foundation. (n.d.). *http.server: HTTP servers.* Python 3.12 documentation. Retrieved November 4, 2026, from https://docs.python.org/3.12/library/http.server.html

Selenium. (n.d.). *WebDriver.* Retrieved November 4, 2026, from https://www.selenium.dev/documentation/webdriver/

Soltani, M., Hermans, F., & Bäck, T. (2020). The significance of bug report elements. *Empirical Software Engineering, 25*(6), 5255–5294. https://doi.org/10.1007/s10664-020-09882-z

Appendix A. Supporting evidence and review references

Assessed analysis is in the main report. The following links preserve supporting activities and raw results; all repository links require assessor access.

Requirements and traceability

I1 draft PR #8

I2 draft PR #10

I1 checks and logs

I2 checks: actual browser gate

Prospective PI-01 plan

I1/I2 mutation evidence

Raw performance and fingerprints

Original-resolution product screenshots

Demo and individual statement templates

Fixed technical review candidate

Reproducible source snapshot

I1: cee24aaa908991a9ff38967bfc2a46f973493934. I2 implementation and performance measurement: f36dfc792c4467c0726598cce401629f7a9e7fe5. The later review tag adds the report and evidence archive; source hashes identify the measured application. Final assessed submission reference must follow the team's actual independent approval.

Assistance and outstanding activities

Codex assisted with research synthesis, implementation, tests, technical experiments, documentation and this draft through the authorised owner's account. These materials do not establish four students' contributions or independent peer review. The team must verify the material, apply course rules for assistance, complete genuine activities and replace pending identities/statements before submission.